

02 Credit Card Fraud Detection

신용카드 사기 거래 탐지 — 이진 분류, 초극단적 클래스 불균형

2.1 프로젝트 개요

28개의 PCA(주성분분석) 변환 변수(V1~V28)와 거래 시각(Time), 거래 금액(Amount)으로 **사기 거래 여부(Class)**를 예측하는 이진 분류 문제다. 담당 모델은 **RandomForest + SMOTE**이며, 임계값(threshold)을 재현율 중심으로 조정해 준 것이 최종 채택안이다. 이 과정에서 언더샘플링·SMOTE·SMOTE-ENN 세 샘플링 기법의 순수한 효과를 비교하는 사전 실험도 함께 진행했다(\$2.6.4).

284,807

전체 거래 건수

31

컬럼 수 (Time+V1~V28+Amount+Class)

0.173%

사기(Class=1) 비율

0건

결측치

2.1.1 입력 피쳐 구성

#	컬럼명	설명	타입
1	Time	데이터셋 내 첫 거래 이후 경과 시간(초). 관측 구간은 총 48시간 (172,792초)	Numeric
2~29	V1 ~ V28	원본 거래 정보를 PCA로 변환한 28개의 익명 연속형 변수	Numeric(PCA)
30	Amount	거래 금액(달러). \$0 ~ \$25,691.16	Numeric
—	Class	예측 대상. 0=정상 거래, 1=사기 거래	Target(Binary)

card_df.info() 기준 전체 31개 컬럼 모두 결측 없이 float64(30개)·int64(1개, Class)로 채워져 있다.

2.1.2 EDA 요약 통계

\$88.35

Amount 평균

\$22.00

Amount 중앙값

\$25,691

Amount 최댓값

67.4 MB

메모리 사용량

평균(\$88.35)이 중앙값(\$22.00)보다 4배 가까이 크다는 것은 소액 거래가 대부분이고 극소수의 고액 거래가 평균을 끌어올리는 **오른쪽으로 긴 꼬리(right-skewed)** 분포임을 보여준다. 실제로 Amount의 왜도(skewness)는 **16.98**로 극단적으로 크며, 이는 §2.3에서 로그 변환을 적용하는 근거가 된다.

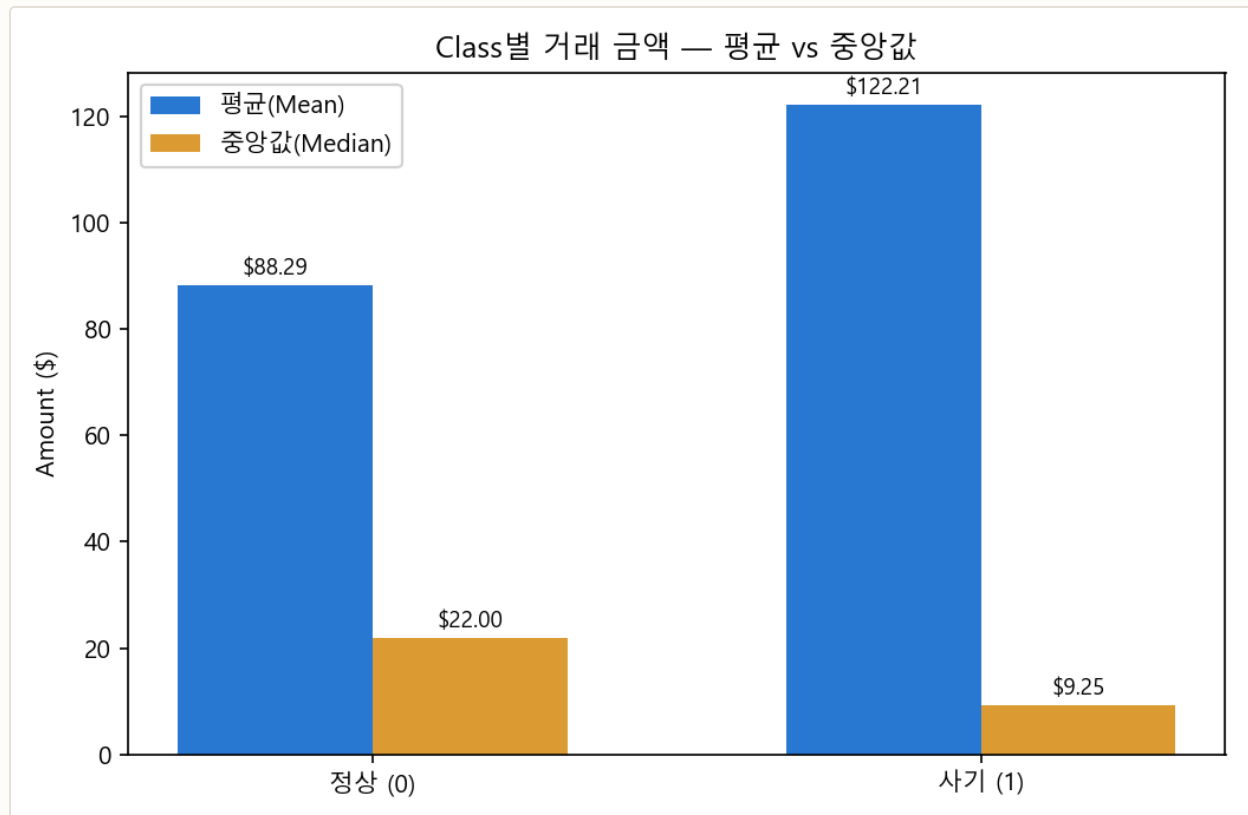


그림 2-1. Class별 거래 금액의 평균과 중앙값. 사기 거래는 평균이 정상 거래보다 높지만(+38%) 중앙값은 오히려 낮다 (-58%).

평균과 중앙값이 반대로 움직이는 이유. 사기 거래의 평균 금액(\$122.21)은 정상 거래(\$88.29)보다 높지만, 중앙값은 사기(\$9.25)가 정상(\$22.00)보다 오히려 낮다. 이는 사기 거래 금액의 분포가 **양극화**돼 있음을 뜻한다 — 카드 정보 탈취 초기에 소액 결제로 카드가 살아있는지 확인하는 "테스트 결제"가 다수 섞여 있고, 동시에 일부 고액 사기가 평균을 끌어올린다. 소액 거래라고 해서 안전하다고 볼 수 없다는 뜻이며, 이 특성 때문에 Amount 자체보다 V1~V28의 패턴을 종합적으로 학습하는 모델(RandomForest)이 금액 기준 규칙보다 효과적이다.

2.1.3 Feature 이름 특징

V1~V28은 원본 거래 정보(가맹점, 위치, 카드 종류 등 민감 정보를 포함할 수 있는 항목)를 PCA로 변환한 익명 변수다. PCA 축이므로 이름 자체에는 의미가 없지만, PCA의 정의상 **서로 직교(orthogonal)하도록** 구성된다는 성질은 보장된다 — 즉 V1~V28 사이에는 구조적으로 완전한 선형 중복이 나타날 수 없다. 이는 Santander의 `ind_*/num_*/saldo_*` 접두어 체계처럼 사람이 읽을 수 있는 이름 패턴과 대비된다. `Time` 과 `Amount` 만 PCA를 거치지 않은 원본 변수다.

2.1.4 동일 값인 Feature

`card_df.describe().loc["std",:].unique()` 의 결과 31개 컬럼 모두 서로 다른 표준편차 값을 가졌다 — **표준편차가 0인(=모든 행이 같은 값인) 컬럼은 하나도 없다**. Santander에서 분산 0인 컬럼이 34개나 나온 것(§2.1.5)과 대조적이다. PCA는 정보가 없는(분산이 거의 0에 가까운) 축은 애초에 주성분으로 채택하지 않으므로, 이 데이터셋에서는 구조적으로 무의미한 상수 컬럼이 만들어지지 않는다.

2.1.5 타깃 변수(Class) 분포 분석

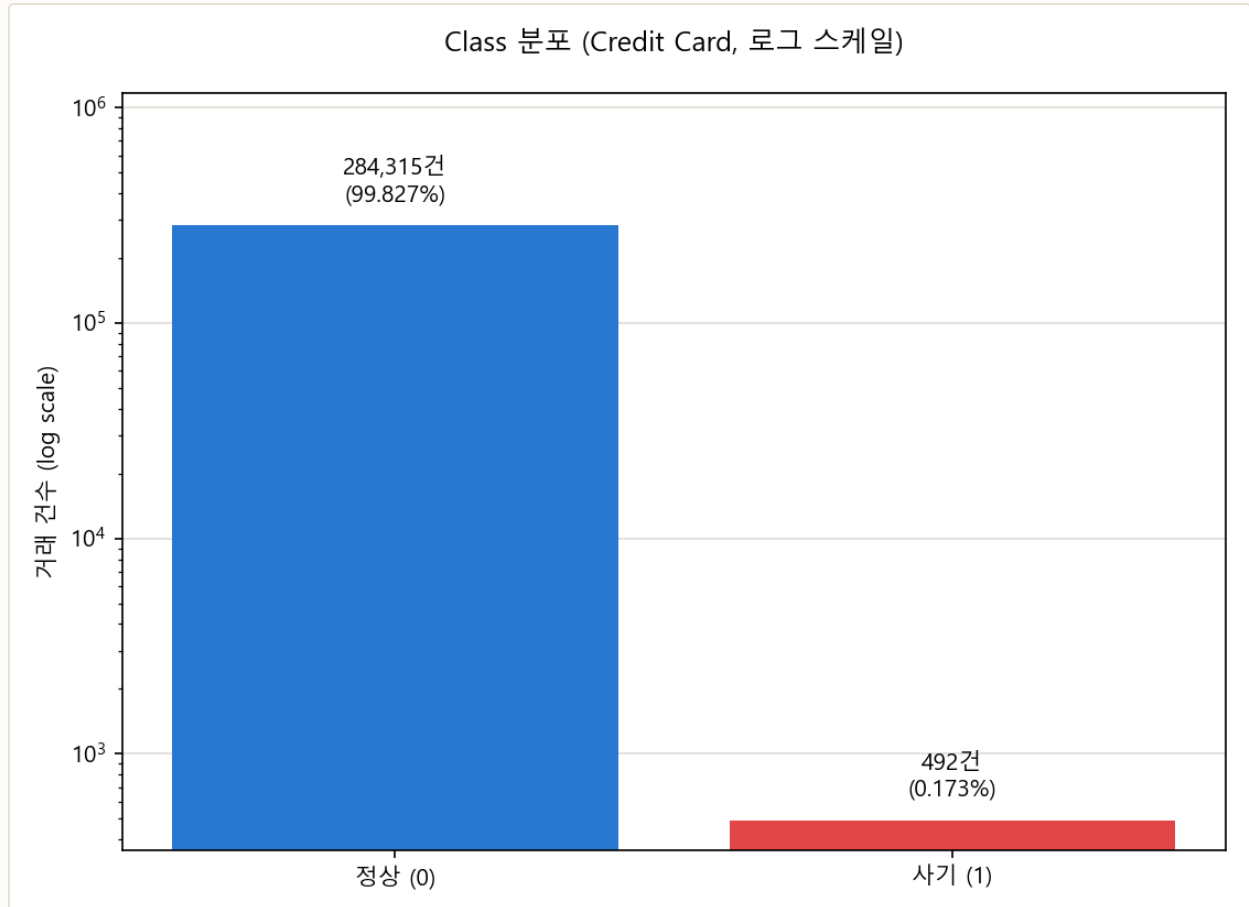


그림 2-2. Class 분포(로그 스케일) — 정상 284,315건(99.827%) vs 사기 492건(0.173%). 약 578:1 비율.

해석. Santander(24:1)보다 100배 이상 심한 불균형이다. 이 정도 비율에서는 ROC-AUC조차 낙관적으로 보일 수 있다 — 음성(정상) 클래스가 압도적으로 많으면 거짓 양성 비율(FPR)의 분모가 매우 커져, 어지간한 오분류로는 FPR이 잘 오르지 않기 때문이다. 그래서 이 프로젝트에서는 ROC-AUC와 함께 AUPRC(정밀도-재현율 곡선의 면적)를 함께 본다(§2.2).

2.1.6 변수 간 상관관계

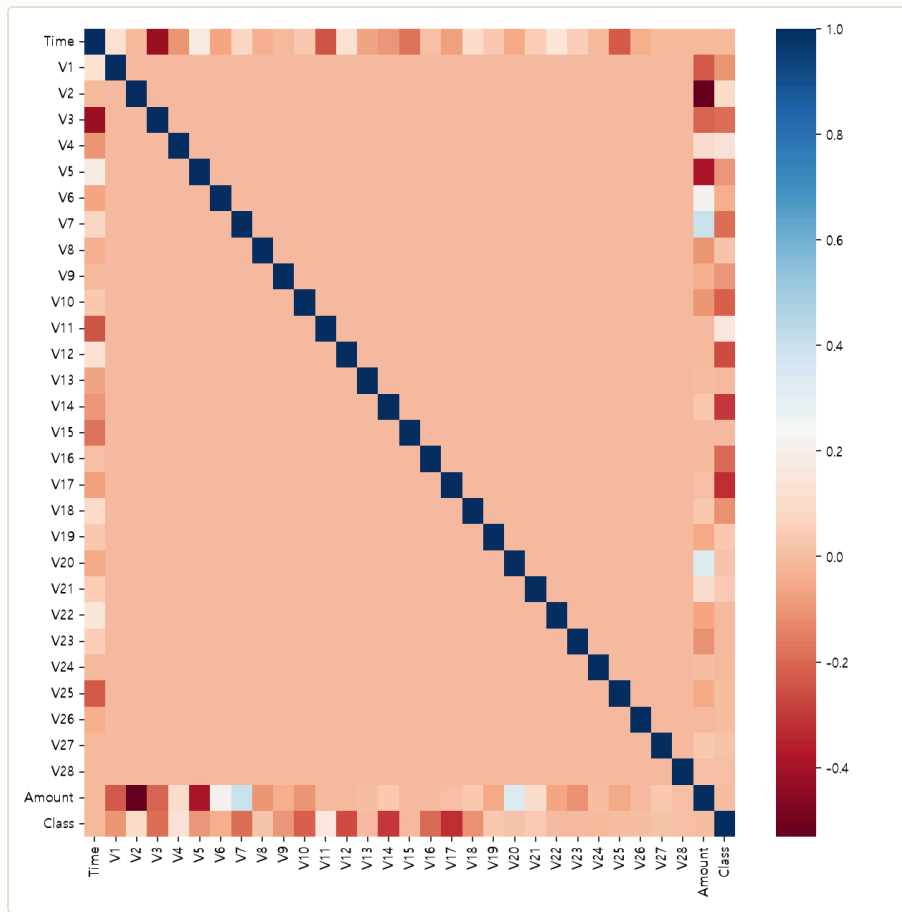


그림 2-3. 31개 변수 전체의 피어슨 상관관계 히트맵(card_df.corr(), cmap='RdBu'). 파란색이 양의 상관, 빨간색이 음의 상관이다. V1~V28 블록이 대각선만 진하고 나머지는 균일한 것은 PCA 축들이 서로 직교하기 때문이며(§2.1.3), 실제로 V1~V28 사이의 상관관계 최댓값은 $|r|=0.0000$ 이다. 색이 뚜렷하게 갈리는 곳은 맨 위 Time 행, 아래 Amount 행, 맨 아래 Class 행뿐이다.

전처리 대상은 V14(-0.303)다. 상관관계가 큰 V14의 이상치를 제거하려 한다. 상관관계가 모두 음수인 상위 변수(V17~V14~V12~V10 등)가 많다는 것은, 이 PCA 축들에서 **값이 작을수록 사기 확률이 높아지는 방향**으로 주성분이 정렬되어 있다는 뜻이다.

2.1.7 거래 시각(Time) 패턴

Time 은 첫 거래 이후 경과 초이므로, 172,792초(48시간)를 24시간 주기로 환산($(\text{Time}/3600) \% 24$) 하면 하루 중 어느 시간대에 거래가 몰리는지 볼 수 있다. 코드에는 없는 분석이지만, Time 컬럼이 왜 §2.3에서 제거되는지 근거를 보태기 위해 직접 확인했다.

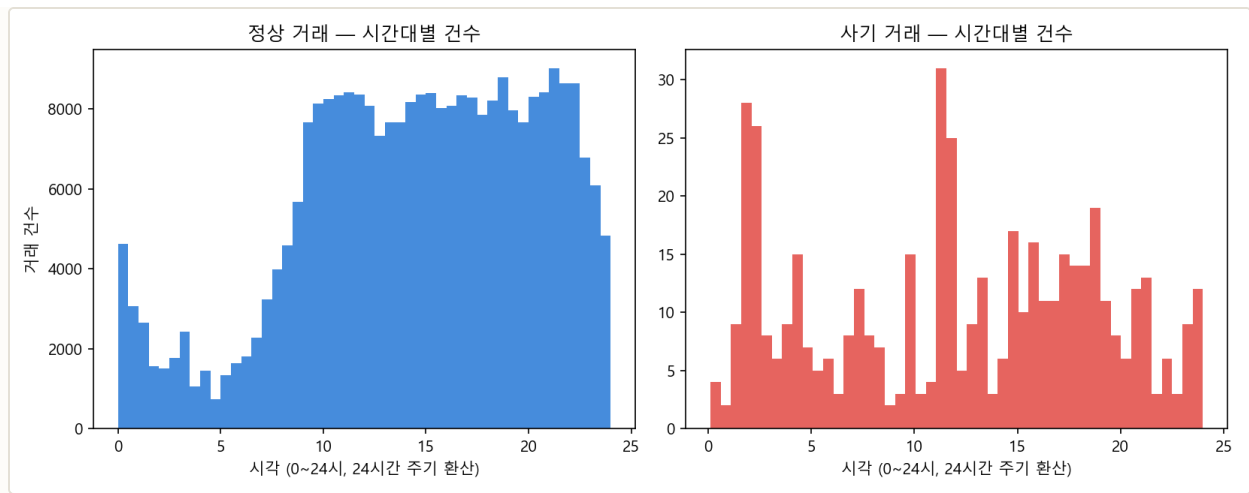


그림 2-4. 시간대별 거래 건수 — 정상 거래(왼쪽)는 새벽 0~6시에 뚜렷이 감소하지만, 사기 거래(오른쪽)는 이 시간대에도 상대적으로 활발하다.

새벽 시간대 사기 비중이 3배 높다. 정상 거래 중 새벽 0~6시에 발생한 비율은 8.37%인 반면, 사기 거래 중 같은 시간대 비율은 25.20%다 — 카드 소유자가 잠든 시간에 도난·탈취된 카드 정보로 거래를 시도하는 패턴과 부합한다. 다만 이 신호를 **Time 자체를 피쳐로 남겨 활용하지 않은 이유**는, 이 값이 "데이터 수집 시작 시점으로부터 경과한 초"일 뿐 실제 시계(time-of-day)가 아니어서 절대적인 해석에 한계가 있고, 이미 V1~V28 중 몇몇 변수가 시간 관련 정보를 간접적으로 포함하고 있을 가능성이 있기 때문이다(\$2.3 참고). 이 패턴은 어디까지나 참고용 EDA 관찰이며 전처리 근거를 보강하는 용도로 남긴다.

2.2 결과 지표

2.2.1 결과 지표 정의 및 선정 근거

지표	정의	이 데이터에서의 역할
Recall(재현율)	실제 사기 중 모델이 찾아낸 비율	최우선 지표 — 놓친 사기는 그대로 금전 손실
Precision(정밀도)	사기로 예측한 것 중 실제 사기 비율	너무 낮으면 정상 거래 차단·고객 불편 급증
F1-Score	정밀도·재현율의 조화평균	재현율만 극단적으로 높이는 것을 견제하는 균형 지표
ROC-AUC	전체 임계값에서의 TPR-FPR 트레이드오프	모델 판별력 비교(단독으로는 과대평가 위험, \$2.1.5)
AUPRC	Precision-Recall 곡선의 면적	극단적 불균형에서 ROC-AUC를 보완하는 핵심 지표

이 프로젝트의 핵심 의도 — 재현율 우선

정밀도와 재현율 중에서는 **재현율을 우선했다**. 카드 사기가 실제로 발생했는데도 모델이 이를 탐지하지 못하는 경우(False Negative)는 그대로 금전 피해로 이어지지만, 정상 거래를 사기로 오탐지하는 경우(False Positive)는 고객에게 확인 연락이 한 번 더 가는 정도의 불편에 그친다. **오탐이 늘더라도 사기를 놓치지 않는 쪽을 택한다**는 것이 \$2.5~2.6에서 임계값을 낮춰 재현율을 끌어올리는 이유다.

이 다섯 지표는 모두 **혼동행렬(Confusion Matrix)**에서 계산된다. 이 데이터셋의 맥락으로 혼동행렬의 네 칸을 다시 정리하면 다음과 같다.

예측: 정상		예측: 사기	
실제: 정상	TN - 정상을 정상으로. 문제 없음	FP - 정상을 사기로 오탐. 고객 재확인 비용	
실제: 사기	FN - 사기를 정상으로. 탐지 실패, 금전 손실	TP - 사기를 사기로. 탐지 성공	

$Recall = TP / (TP + FN)$ 이므로, 재현율을 높인다는 것은 결국 **FN(탐지 실패)을 줄이는 것과 동의어**다. 반대로 $Precision = TP / (TP + FP)$ 는 FP(오탐)를 줄일수록 올라간다. 두 지표가 동시에 최고가 되는 임계값은 일반적으로 존재하지 않으므로(\$2.6.2에서 실제로 확인), 이 프로젝트에서는 FN을 줄이는 쪽에 가중치를 둔 임계값을 선택했다.

2.2.2 결과 지표 해석 계획

하이퍼파라미터 튜닝(GridSearchCV)의 기준 지표로는 accuracy 대신 **Recall(재현율), ROC-AUC** 를 사용했다 — 이 데이터처럼 양성(사기) 비율이 0.173%뿐인 경우 accuracy는 항상 99.8% 이상으로 나와 모델 간 우열을 가리는 데 도움이 되지 않기 때문이다. 임계값 조정은 모델 학습이 끝난 뒤 별도 단계로 진행하므로, 튜닝 단계에서는 임계값에 의존하지 않는 지표(ROC-AUC)로 샘플링 기법과 모델의 우열을 먼저 가리고, 최종 임계값은 재현율 목표에 맞춰 \$2.6에서 따로 정한다.

2.3 데이터 전처리 & 클리닝

처리	내용	근거
Time 제거	거래 순번성 정보로 사기 여부와 직접 관련 없음	노이즈 제거
Amount 로그 변환	$\log_{10}(\text{Amount})$ → Amount_Scaled 컬럼 추가	왜도 16.98 → 0.16로 완화(그림 2-5)
V14 이상치 제거	사기(Class=1) 데이터의 V14 IQR×1.5 범위 밖 값 제거	사기 클래스 내에서 상관관계 상위권 변수의 잡음 축소
train/test 분할	7:3, <code>stratify=y</code>	0.173% 비율을 양쪽 세트에 동일하게 유지

```
def get_preprocessed_df(df):
    df_copy = df.copy()
    amount_n = np.log1p(df_copy['Amount'])
    df_copy.insert(0, 'Amount_Scaled', amount_n)
    df_copy.drop(['Time', 'Amount'], axis=1, inplace=True)
    return df_copy

# train_test_split(test_size=0.3, random_state=42, stratify=y_target) 이후,
# train과 test 각각에서 독립적으로 V14 이상치를 제거한다.
```

log1p 변환 전후 Amount 통계 비교

구분	평균	표준편차	왜도(skew)
변환 전 (Amount, \$)	88.35	250.12	16.98
변환 후 (Amount_Scaled = log1p)	-	-	0.16

왜도(skewness)는 0에 가까울수록 좌우 대칭에 가깝다. 16.98은 정규분포(왜도 0)와 비교할 수 없을 만큼 큰 값이며, 로그 변환만으로 0.16까지 낮아진다는 것은 Amount의 비대칭성이 극소수 고액 거래에 의해 만들어진 것이지 데이터 자체의 근본적인 특성이 아니라는 뜻이다.

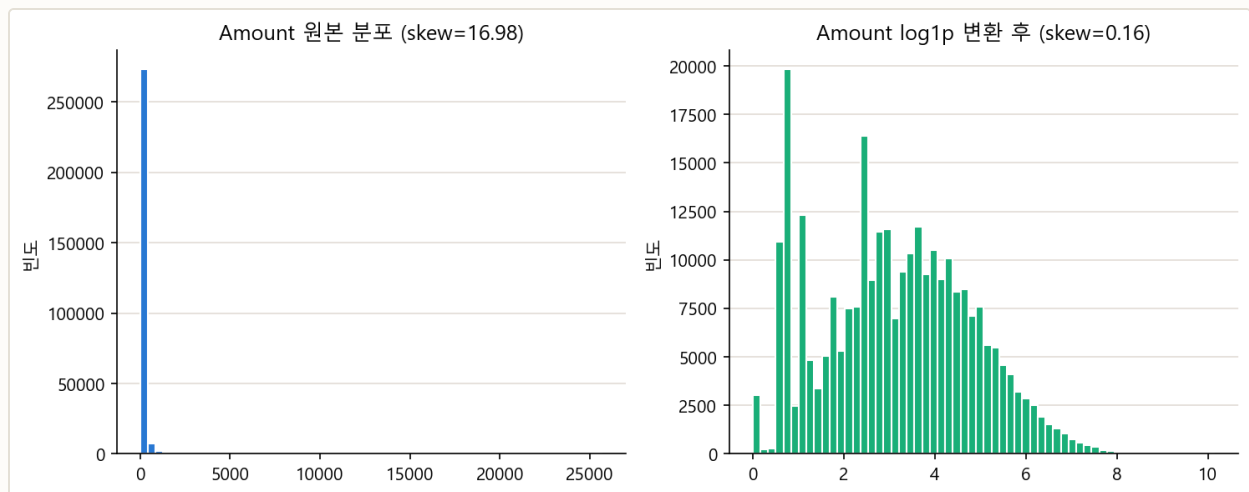


그림 2-5. Amount 원본(왜도 16.98) vs log1p 변환 후(왜도 0.16). 변환 후 다봉형(multi-modal)이지만 극단적 꼬리는 크게 줄었다.

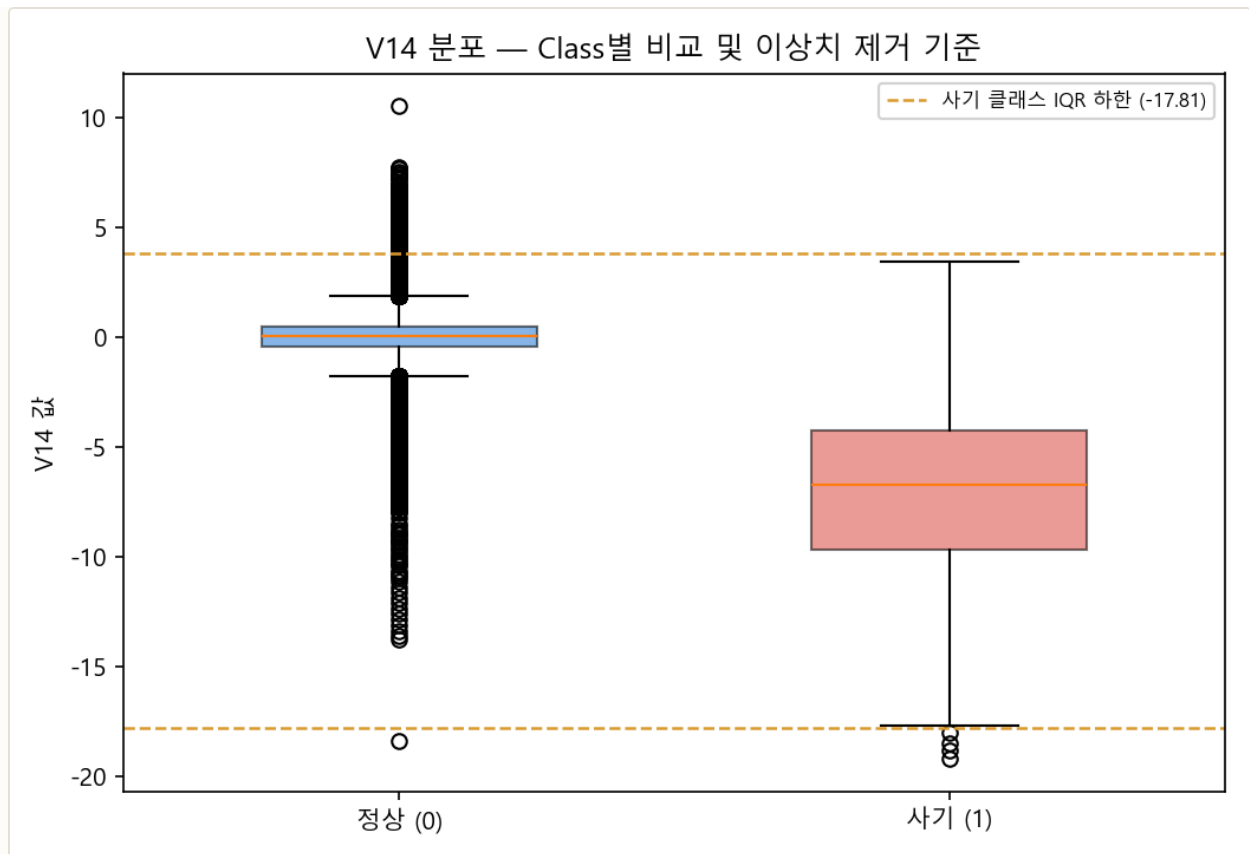


그림 2-6. V14 분포의 Class별 비교. 사기 클래스(오른쪽)는 정상 클래스(왼쪽)보다 값이 뚜렷이 낮은 쪽으로 이동해 있으며, 점선은 사기 클래스 기준 IQR×1.5 이상치 경계(-17.81 ~ 3.83)다.

V14가 이상치 제거 대상으로 적합한 이유가 시각적으로 드러난다. 정상 거래(파란 박스)는 중앙값이 0 부근에 밀집해 있는 반면, 사기 거래(빨간 박스)는 중앙값이 약 -6.8로 뚜렷하게 낮은 쪽에 분포한다 — 두 클래스의 분포가 잘 분리되어 있어 V14가 사기 판별에 유용한 변수임을 보여준다. 다만 사기 클래스 내부에서도 IQR 기준 하한(-17.81)보다 더 낮은 값 몇 건이 존재하며, 이 극단값들이 학습 시 결정경계를 왜곡할 수 있어 §2.3의 이상치 제거 대상이 된다.

2.4 피처 구조화 & 엔지니어링

V1~V28은 이미 PCA로 압축된 피처라 추가 파생 변수를 만들 여지가 크지 않다. 이 노트북에서 유일하게 새로 만든 컬럼은 §2.3의 `Amount_Scaled` (로그 변환된 Amount)이며, 그 외 범주형 인코딩이나 텍스트 벡터화도 필요 없다 — 31개 컬럼이 모두 이미 수치형이기 때문이다. 이 프로젝트에서 실질적으로 "피처 엔지니어링"에 해당하는 작업은 **클래스 불균형을 보정하는 샘플링 전략 선택**이며, 이는 모델링과 결합되므로 §2.5에서 함께 다룬다.

2.5 모델링 & 하이퍼파라미터 튜닝

2.5.1 왜 리샘플링이 필요한가

대부분의 분류 모델은 학습 과정에서 **전체 손실(loss)을 최소화**하는 방향으로 학습한다. 사기 비율이 0.173%인 원본 데이터를 그대로 학습하면, 모델이 사기 492건을 모두 틀리게 예측해도 손실 총합에

는 거의 영향이 없다 — 정상 284,315건만 잘 맞으면 손실이 이미 충분히 낮기 때문이다. 그 결과 모델은 "사기 패턴을 배우는 것"보다 "다수 클래스를 더 정확히 맞히는 것"에 최적화되기 쉽다. 리샘플링은 학습 단계에서 두 클래스의 비중을 강제로 맞춰, 모델이 사기 패턴 자체를 무시하지 못하게 만드는 장치다.

2.5.2 샘플링 전략 3종

기법	방식	학습 데이터 변화
언더샘플링	정상 거래를 사기 건수만큼 무작위로 축소	199,362건 → 684건 (342 / 342)
SMOTE	사기 샘플 주변에 합성 데이터를 생성해 오버샘플링	199,362건 → 398,040건 (199,020 / 199,020)
SMOTE-ENN	SMOTE 이후 경계가 모호한 샘플을 ENN으로 추가 제거	199,362건 → 397,717건 (199,020 / 198,697)

세 기법 모두 **train 데이터에만** 적용했다(test는 원본 분포 유지). 세 기법의 순수한 효과 비교 결과는 §2.6.4 (1)에서 다룬다. 최종적으로 5개 모델 모두 **SMOTE**를 채택했다 — 언더샘플링은 정상 거래 정보를 99.7% 가까이 버려 실무 적용이 어렵고(§2.6.4), SMOTE-ENN은 SMOTE 대비 뚜렷한 이점이 크지 않았기 때문이다.

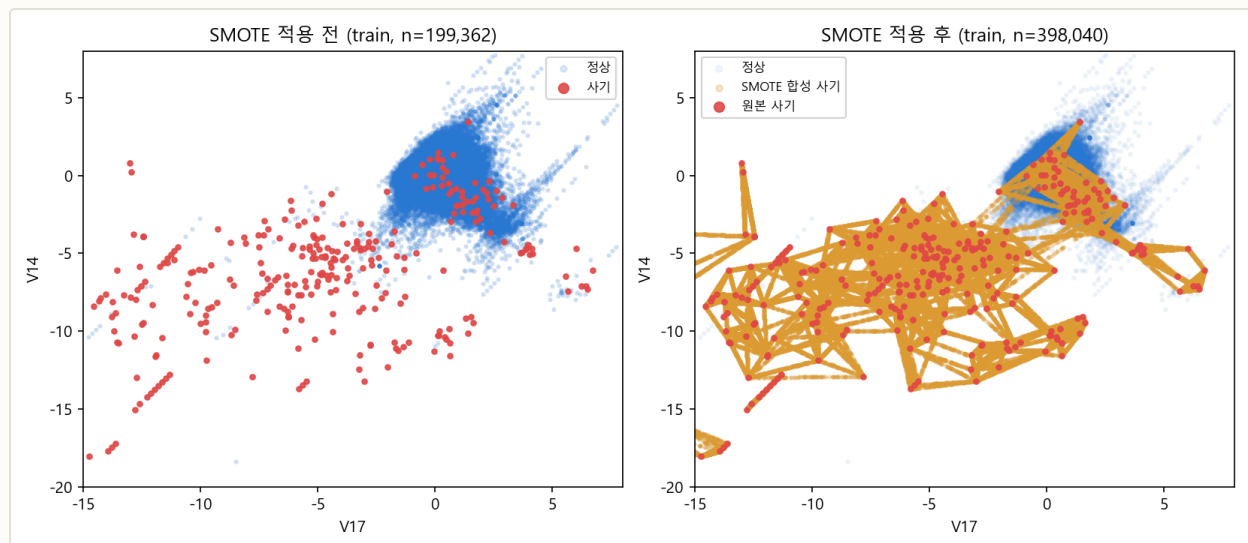


그림 2-7. 상관관계가 가장 큰 두 변수(V17, V14) 평면에 투영한 SMOTE 적용 전후 비교. 왼쪽은 원본 사기 샘플(빨강)이 정상 샘플(파랑)에 둘러싸여 희소하게 흩어져 있고, 오른쪽은 원본 사기 샘플 사이를 잇는 선분 위에 합성 샘플(금색)이 채워진 모습이다.

SMOTE가 실제로 하는 일이 이 그림에 그대로 나타난다. SMOTE(Synthetic Minority Oversampling TEchnique)는 소수 클래스(사기) 샘플 하나를 골라 **가장 가까운 이웃 사기 샘플과의 사이를 잇는 선분 위의 임의의 지점**에 새 합성 샘플을 만든다. 오른쪽 그림에서 금색 점들이 빨간 원본 사기 점들을 잇는 그 물망 형태로 나타나는 것이 바로 이 보간(interpolation) 과정이다. 이 방식 덕분에 SMOTE는 언더샘플링과 달리 정상 거래 정보를 하나도 버리지 않으면서도, 모델이 학습할 수 있는 사기 샘플의 절대량을 199,020건 (원본 342건의 약 582배)까지 늘릴 수 있다. 다만 이 보간은 **기존 사기 샘플들이 이루는 공간**

안에서만 이루어지므로, 원본 사기 샘플이 커버하지 못하는 완전히 새로운 유형의 사기 패턴까지 만들어 내지는 못한다는 한계도 동시에 보여준다.

2.5.3 GridSearchCV 탐색 (RandomForest 기준)

최종 채택 모델인 **RandomForest**를 대상으로, §2.2.2에서 정한 대로 임곗값에 의존하지 않는 `scoring='roc_auc'`를 기준으로 5-fold 교차검증 탐색을 수행했다. 세 개의 구조적 파라미터를 각각 2개 값씩, 총 8개 조합 × 5폴드 = 40회 학습이다.

```
smote_pipeline = ImbPipeline([
    ('sampler', SMOTE(random_state=42)),
    ('classifier', RandomForestClassifier(n_jobs=-1, random_state=42))
])
rf_smote_grid = GridSearchCV(smote_pipeline, param_grid={
    'classifier__n_estimators': [300, 1000], 'classifier__max_depth': [None, 16],
    'classifier__min_samples_leaf': [1, 3]
}, cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42), scoring='roc_auc')
# Best Params: n_estimators=1000, max_depth=None, min_samples_leaf=1
# Best CV ROC-AUC: 0.9811
```

8개 조합의 교차검증 ROC-AUC (평균 ± 표준편차, 순위순)

n_estimators	max_depth	min_samples_leaf	CV ROC-AUC	표준편차	순위
1000	None	1	0.9811	0.0071	1
1000	16	3	0.9810	0.0081	2
1000	None	3	0.9806	0.0076	3
1000	16	1	0.9806	0.0091	4
300	16	3	0.9802	0.0072	5
300	16	1	0.9792	0.0112	6
300	None	3	0.9791	0.0083	7
300	None	1	0.9761	0.0103	8

탐색한 RandomForest 파라미터가 실제로 조절하는 것

파라미터	조절 대상	이 데이터에서의 선택
n_estimators	양상블에 포함되는 결정 트리의 개수. 많을수록 확률 추정이 부드러워지지만 학습·예측 시간이 비례해서 늘어난다	1000(탐색 범위 중 큰 값) 선택 — 상위 4개 조합이 모두 1000이었다. 트리 수가 이 데이터에서 가장 영향이 큰 파라미터였다

파라미터	조절 대상	이 데이터에서의 선택
max_depth	트리의 최대 깊이. None은 잎이 순수해질 때까지 제한 없이 분기함을 뜻한다	None(무제한) 선택 — 다만 16으로 제한한 조합(0.9810)과의 차이가 0.0001에 불과해, 사실상 성능 차이가 없었다
min_samples_leaf	잎 노드가 가져야 하는 최소 샘플 수. 값을 키우면 트리가 덜 자라 과적합이 억제된다	1(기본값) 선택 — SMOTE로 사기 샘플을 199,020건까지 늘려둔 덕분에 잎을 끝까지 쪼개도 과적합이 문제되지 않았다

이 프로젝트에서는 하이퍼파라미터로 인한 개선이 크지 않아 RandomForest의 구조적 하이퍼파라미터를 더 파고드는 대신 **분류 임계값을 재현율 목표에 맞게 조정하는 쪽**에 튜닝 자원을 집중했다 (§2.6.2).

2.6 성능 검증 결과

2.6.1 파이프라인 요약 다이어그램



2.6.2 지표 결과 (채택 파이프라인: RandomForest + SMOTE)

동일한 모델을 세 가지 임계값에서 평가해, 임계값 조정이 성능에 미치는 영향을 그대로 보여준다.

임계값	Accuracy	Precision	Recall	F1	ROC-AUC	AUPRC
0.500 (기본값)	0.9994	0.8712	0.7823	0.8244	0.9854	0.8314
0.727 (최대 F1 지점)	-	0.9558	0.7347	0.8308	0.9854	0.8314
0.305 (채택 — 재현율 우선)	0.9992	0.7500	0.8367	0.7910	0.9854	0.8314

채택 임계값(0.305) 혼동행렬: TN=85,254 · FP=41 · FN=24 · TP=123 (테스트셋 사기 147건 중 123건 탐지). ROC-AUC와 AUPRC는 임계값에 무관한 확률 기반 지표라 세 행에서 값이 동일하다.

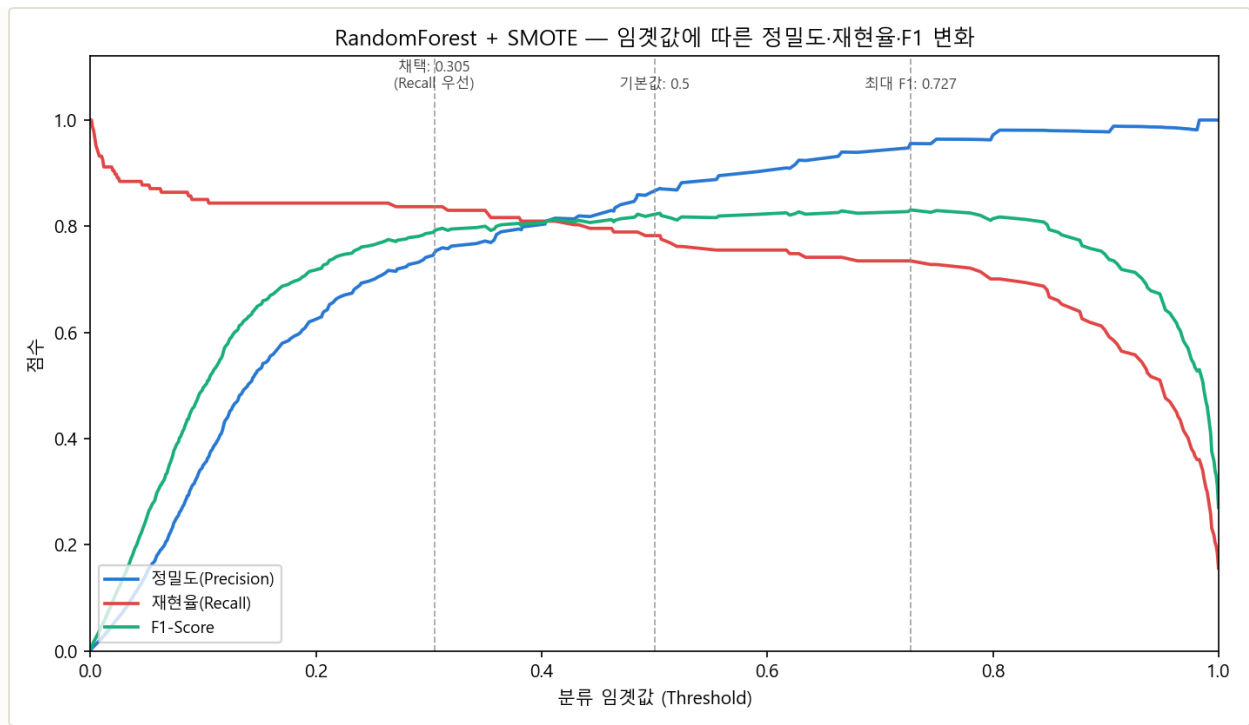


그림 2-8. RandomForest + SMOTE의 임계값별 정밀도·재현율·F1 변화. 임계값을 0.727 → 0.305로 낮출수록 재현율은 오르고 정밀도는 떨어진다.

2.6.3 지표 해석

왜 최대 F1(0.8308) 지점 대신 0.305를 골랐는가. 임계값 0.727에서는 F1이 가장 높지만(0.8308) 재현율은 0.7347에 그친다 — 테스트셋 사기 147건 중 **39건을 놓친다**는 뜻이다. 임계값을 0.305로 낮추면 F1은 0.7910으로 5.9%p 낮아지지만, 재현율은 0.8367로 올라 **놓치는 사기가 24건까지 줄어든다**(39건 → 24건, 15건 추가 탐지). 이 프로젝트는 §2.2에서 밝힌 대로 "F1을 최대화하는 것"이 아니라 "F1이 크게 나빠지지 않는 선에서 재현율을 최대화하는 것"을 목표로 했으므로, F1 하락폭을 최대 F1 대비 5% 이내로 제한하는 조건에서 재현율이 가장 높은 임계값을 탐색해 0.305를 선택했다.

이 조정의 대가는 정밀도다. 정밀도는 0.9558(임계값 0.727)에서 0.7500(임계값 0.305)으로 떨어진다 — 사기로 예측한 거래 4건 중 1건은 실제로는 정상 거래라는 뜻이다. 다만 §2.2에서 정의한 대로 오탐지의 비용(고객 재확인 연락)은 탐지 실패의 비용(금전 손실)보다 훨씬 작다고 보았으므로, 이 트레이드오프는 이 프로젝트의 목적에 부합한다.

구체적인 활용 예시. 이 테스트셋의 사기 거래 147건의 평균 금액은 §2.1.2에서 확인한 Class별 평균을 그대로 적용하면 **건당 약 \$122**다. 임계값 0.727을 쓰면 39건(약 \$4,760 상당)을 놓치지만, 0.305를 쓰면 24건(약 \$2,930 상당)만 놓친다 — 채택한 임계값이 약 **\$1,830여치의 사기를 추가로 막아낸다는** 뜻이다. 그 대가로 정상 거래 41건이 추가로 오탐지되어 고객에게 확인 연락이 가지만, 이는 카드사가 통상적으로 감내할 수 있는 운영 비용 수준이다.

ROC-AUC(0.9854)만 보면 매우 높아 보이지만, AUPRC(0.8314)는 상대적으로 낮다 — 이는 §2.1.5에서 설명한 대로 극단적 불균형에서 ROC-AUC가 실제보다 낙관적으로 보이는 현상을 보여주는 구체적 사례다. **이 데이터셋에서는 AUPRC가 ROC-AUC보다 모델 성능을 더 보수적이고 현실적으로 보여준다.**

2.6.4 모델별 결과 비교 및 추론

세 개의 관점 — 샘플링 기법, 모델 종류, 임곗값 — 을 각각 독립적으로 비교해, 최종 채택안(RandomForest + SMOTE + 임곗값 0.305)이 세 관점 모두에서 근거를 갖췄는지 확인한다.

(1) 샘플링 기법별 비교

기본 파라미터로 통일해 세 기법만 바꿔 비교했다.

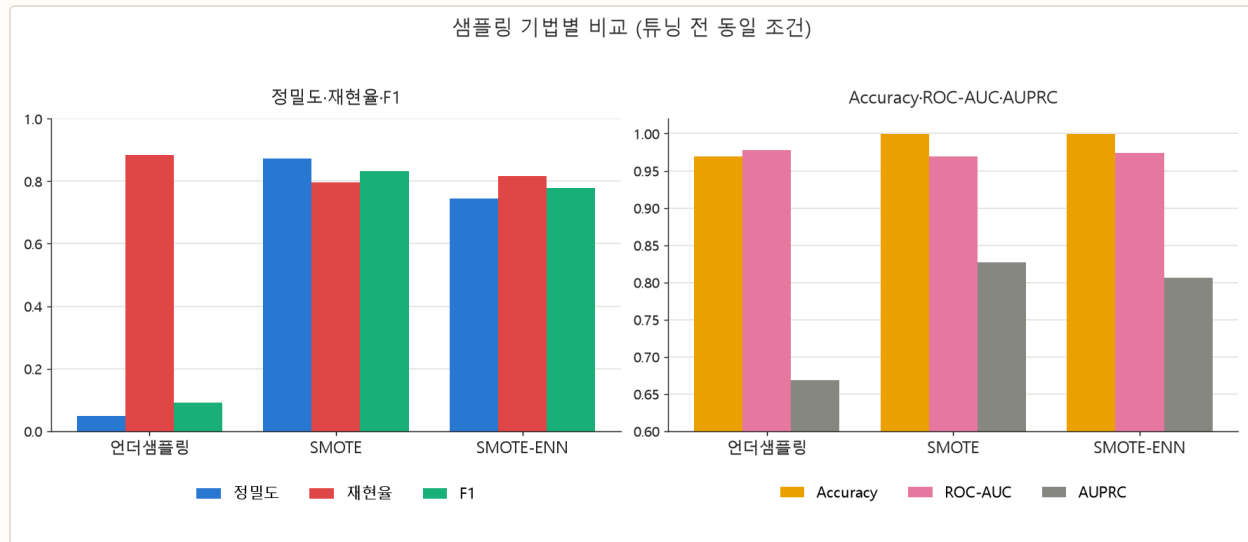


그림 2-9. 샘플링 기법별 비교(튜닝 전 동일 조건).

샘플링	Accuracy	Precision	Recall	F1	AUC	AUPRC
언더샘플링	0.9695	0.0478	0.8844	0.0906	0.9777	0.6689
SMOTE	0.9994	0.8731	0.7959	0.8327	0.9697	0.8274
SMOTE-ENN	0.9992	0.7453	0.8163	0.7792	0.9739	0.8059

언더샘플링은 왜 재현율이 가장 높은데(0.8844) 정밀도가 극단적으로 낮은가(0.0478). 학습 데이터를 684건까지 줄이면서 정상 거래 정보 대부분(199,362건 중 198,678건)을 버렸다. 모델이 "사기 패턴"을 공격적으로 학습해 재현율은 높지만, 실제 운영에서는 정상 거래 20건 중 1건꼴로 사기 경보가 울리는 셈이라 실용성이 떨어진다. **SMOTE**는 정상 거래 정보를 보존하면서 사기 샘플만 합성으로 늘려, 정밀도(0.8731)와 재현율(0.7959)의 균형이 가장 좋다. **SMOTE-ENN**은 SMOTE보다 재현율은 근소하게 높지만(0.8163) 정밀도가 크게 낮다(0.7453) — 경계 샘플을 추가로 제거하는 ENN 단계가 정상 거래처럼 보이는 사기 샘플까지 학습에서 떨어내며 결정 경계를 사기 쪽으로 더 넓힌 것으로 해석된다. **종합적으로 세 기법 중에서는 SMOTE가 정밀도-재현율 균형이 가장 우수해, 이후 5개 모델 비교(2)에서 공통 전처리로 채택했다.**

(2) 모델별 비교

SMOTE 적용 조건에서 시도한 5개 모델이다.

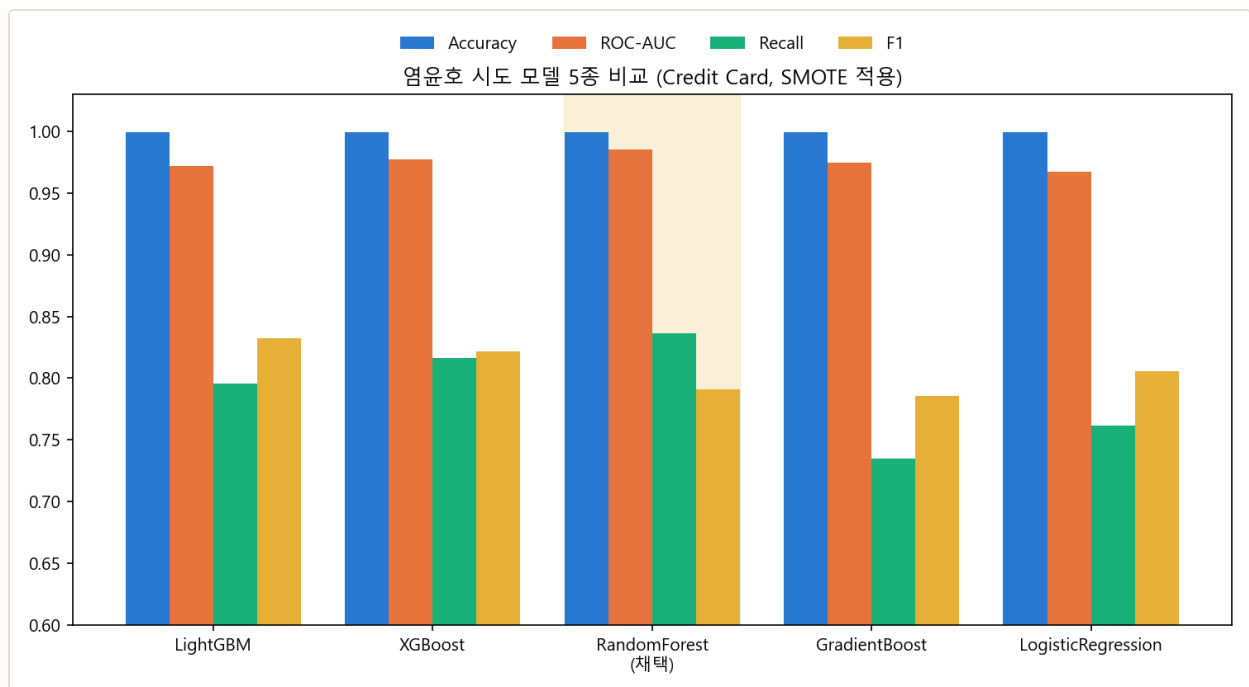


그림 2-10. 모델 5종 비교(SMOTE 적용). 출처: total_result.xlsx.

모델	Accuracy	Precision	Recall	F1	ROC-AUC	비고
LightGBM	0.9994	0.8731	0.7959	0.8327	0.9720	GridSearchCV 튜닝
XGBoost	0.9993	0.8276	0.8163	0.8219	0.9771	기본 파라미터
RandomForest (채택)	0.9992	0.7500	0.8367	0.7910	0.9854	임계값 0.305 조정
GradientBoost	0.9993	0.8438	0.7347	0.7855	0.9749	임계값 0.9844 조정
LogisticRegression	0.9994	0.8550	0.7619	0.8057	0.9675	임계값 최적화

RandomForest를 채택한 근거. ROC-AUC 0.9854로 5개 모델 중 가장 높아 모델 자체의 판별력이 가장 우수했고, 기본 임계값(0.5)에서도 정밀도 0.8712·F1 0.8244로 준수한 균형을 보였다. 여기에 재현율 목표에 맞춘 임계값 조정 여지가 다른 모델보다 컸다 — RandomForest는 여러 트리의 확률을 평균화하는 배깅(bagging) 구조라 확률 추정이 부드럽고 안정적이어서, 임계값을 낮춰도 성능이 급격히 무너지지 않는다. 그 결과 F1은 채택한 LightGBM 단독 비교(0.8327)보다 낮아졌지만(0.7910), **재현율(0.8367)은 5개 모델 중 가장 높다** — 이 프로젝트의 핵심 목표(\$2.2)와 가장 잘 맞는 모델이다.

LightGBM·XGBoost는 F1·정밀도가 상대적으로 높지만 재현율은 RandomForest보다 낮다 (0.7959, 0.8163). 두 모델 모두 그래디언트 부스팅 계열로, 손실 함수를 정밀도-재현율의 균형점 근처에서 최소화 하도록 학습이 진행돼 기본 임계값에서 이미 F1이 최적점에 가깝다. 반대로 말하면 **임계값을 낮춰 재현율을 더 끌어올릴 여지가 RandomForest보다 제한적**이라는 뜻이기도 하다.

LogisticRegression-GradientBoost는 재현율이 가장 낮다(0.76, 0.73). 두 모델 모두 선형 결정경계 또는 상대적으로 얇은 트리 위주 구조라, SMOTE로 늘어난 사기 샘플의 국소적(local) 패턴을 트리 앙상블만큼 세밀하게 분리하지 못한 것으로 판단한다. **종합적으로 ROC-AUC가 가장 높고 임곗값 조정으로 재현율을 극대화할 수 있었던 RandomForest + SMOTE가 이 프로젝트의 목적에 가장 부합해 최종 채택했다.**

(3) AUPRC로 다시 본 5개 모델

§2.1.5에서 밝힌 대로 이 데이터셋에서는 ROC-AUC가 실제보다 낙관적으로 보일 수 있어, AUPRC(정밀도-재현율 곡선의 면적) 기준으로 5개 모델을 다시 확인했다.

모델	ROC-AUC	AUPRC
LightGBM	0.9720	0.8342
XGBoost	0.9771	0.8350
RandomForest (채택)	0.9854	0.8314
GradientBoost	0.9749	0.6687
LogisticRegression	0.9675	0.7024

ROC-AUC 1위와 AUPRC 1위가 다른 모델이라는 것이 핵심 관찰이다. AUPRC만 보면 XGBoost(0.8350)와 LightGBM(0.8342)이 RandomForest(0.8314)보다 근소하게 높다 — 극단적 불균형 상황에서 두 지표가 서로 다른 이야기를 할 수 있다는 §2.1.5의 우려가 실제로 나타난 사례다. 다만 그 차이(0.003~0.004)는 크지 않고, RandomForest는 ROC-AUC 우위(0.9854 vs 0.97대)가 더 뚜렷하며 §2.6.2에서 확인했듯 **임곗값 조정으로 재현율을 가장 크게 끌어올릴 수 있는 모델**이었다. 이 프로젝트는 단일 지표 최댓값이 아니라 "재현율 우선"이라는 목표 함수를 기준으로 모델을 골랐으므로, AUPRC가 근소하게 낮다는 이유만으로 RandomForest 채택을 재고할 근거는 아니라고 판단했다.

2장 종합 결론

① 언더샘플링·SMOTE·SMOTE-ENN 중에서는 정상 거래 정보를 보존하는 SMOTE가 정밀도-재현율 균형이 가장 좋았고, ② SMOTE 적용 후 5개 모델을 비교한 결과 RandomForest가 ROC-AUC(0.9854)와 임곗값 조정 여지에서 가장 우수했으며, ③ 최대 F1 임곗값(0.727, 사기 108건 탐지·39건 손실) 대신 재현율 우선 임곗값(0.305)을 채택해 테스트셋 사기 147건 중 **123건(84%)을 탐지**하고 손실을 24건까지 줄였다. 이 과정 전체가 §2.2에서 정의한 "오탐지 비용 < 탐지 실패 비용"이라는 하나의 원칙을 일관되게 따른 결과다.